UDP-Based Zero Touch Provisioning (ZTP) Assignment

Overview

This assignment is designed to evaluate a candidate's practical networking knowledge along with their ability to design and implement a real system. Candidate will build a simplified Zero Touch Provisioning (ZTP) system using UDP as the transport protocol.

This is not a REST or API exercise.The goal of this assignment is to assess a candidate's ability to reason about networking fundamentals, reliability, security, and system behavior, particularly in the context of real-world operational constraints.

Scope & Expectations

• Language: C or Python
• Deliverables: Working code and a README explaining your design choices

Focus on correctness, and networking fundamentals rather than UI or packaging.

Problem Statement

A network device is powered on in the field with no prior configuration. It must automatically discover a provisioning server, establish a secure UDP-based control channel, register itself, receive configuration and firmware, and finally transition into an operational state.

1. Device Boot & Discovery

On startup, the device should:
• Generate or load a soft serial number
• Discover the provisioning server using one of the following:
  – Static IP (acceptable)
  – Broadcast or multicast discovery (bonus)

Example output:
[BOOT] Soft serial generated
[DISCOVERY] Provisioning server found

2. UDP Session Establishment

All communication must use UDP.

Implement a basic session handshake between the device and server. Your protocol must include:
• Sequence numbers
• Timeouts
• Retransmission logic
• A session identifier

3. Reliable Messaging over UDP

Since UDP is unreliable, you must implement reliability at the application level:
• Acknowledgements
• Duplicate detection
• Retransmissions on timeout

You should be able to handle simulated packet loss (10–20%).

4. NAT Awareness & Keepalives

Assume the device may be behind NAT or firewall devices.

Implement:
• Periodic keepalive messages
• Session timeout detection

Explain in your README why this is important for real deployments.

5. Secure Registration

During registration, the device must send:
• Soft serial
• Device metadata
• A nonce

The server must verify authenticity using an HMAC or equivalent mechanism. Replay protection must be implemented.

6. Configuration & Firmware Provisioning

The server must send:
• A configuration payload (JSON or YAML)
• A firmware blob (mock data is acceptable)

You must:
• Transfer data in chunks
• Validate integrity (hash)
• Resume transfers after failure

Do not rely on IP fragmentation.

7. Version Upgrade & Restart

If the firmware version is outdated:
• Simulate an upgrade
• Restart the device process
• Resume provisioning correctly without duplicate registration

8. Validation & Operational Mode

After provisioning, the device should validate:
• Configuration availability
• Firmware version
• Session health

If successful, transition the device to OPERATIONAL state

9. Server Robustness

The provisioning server should be designed to handle multiple devices registering concurrently.

Specifically, the server must be able to:

- Accept and process registration requests from multiple devices at the same time
- Correctly determine and assign the appropriate firmware version and configuration for each device
- Track and store device registration details, including:
    1. Device identifier (soft serial / device ID)
    2. Timestamp of registration
    3. Firmware version assigned
    4. Configuration version assigned
    5. Registration outcome (success or failure)

The server should maintain this information in a simple in-memory data structure or database (a file-backed or in-memory store is sufficient for this assignment).

At the end of the exercise, the server must be able to:

- Dump the contents of this database
- Display a summary of:
- Devices that were successfully registered
- Devices that failed registration
- Firmware and configuration versions associated with each device

Example of a Dump from Server DB:

Device Registration Details

Device ID	Registration Status	Firmware Assigned	Config Version	Registration Time	Notes
ZTP-93AF21	Success	v1.1	v3	2025-01-18 14:32:10	—
ZTP-84BC19	Failed	—	—	2025-01-18 14:33:02	Invalid HMAC
ZTP-71DD08	Success	v1.1	v3	2025-01-18 14:33:45	—
ZTP-55AC90	Success	v1.1	v3	2025-01-18 14:34:10	—
ZTP-19FE77	Success	v1.0	v2	2025-01-18 14:34:52	Legacy device

.

README Requirements

Your README must explain:
1. How reliability is implemented over UDP
2. How replay attacks are prevented
3. Why IP fragmentation is avoided
4. How NAT affects protocol design
5. What happens under packet loss

Notes

This assignment reflects real-world control-plane engineering problems. There is no single correct solution. We are more interested in your reasoning and tradeoffs than feature completeness.

Appendix A: End-to-End Practical Example

This appendix provides a complete, practical walkthrough of how the Zero Touch Provisioning (ZTP) system is expected to behave in a real deployment. The intent is to clarify expectations and provide concrete values and logs to guide your implementation. You do not need to match this output exactly, but your system should follow the same logical flow.

Scenario
A new network device is deployed in the field with no configuration, no certificates, and no stored credentials.
The device is connected to the network and powered on for the first time.

Device Information
Soft Serial : ZTP-93AF21
Model : EDGE-GW-100
Firmware : v1.0

Step 1: Device Boot
The device boots and generates or loads its soft serial number.

[BOOT] Device starting
[BOOT] Soft serial: ZTP-93AF21
[BOOT] Firmware version: v1.0

Step 2: Server Discovery
The device discovers the provisioning server using a static IP.

Provisioning Server : 10.10.10.5
UDP Port : 5555

[DISCOVERY] Using static server IP 10.10.10.5:5555

Step 3: UDP Session Establishment
The device establishes a logical session over UDP.

Device -> HELLO (seq=1, soft_serial=ZTP-93AF21)
Server -> HELLO_ACK (seq=1, session_id=7421)

[UDP] HELLO sent (seq=1)
[UDP] Session established (session_id=7421)

Step 4: NAT Keepalive
To keep NAT mappings alive, periodic keepalives are exchanged.

[NAT] Keepalive sent
[NAT] Keepalive acknowledged

Step 5: Secure Registration
The device registers securely using a nonce and HMAC.

REGISTER:
session_id = 7421
soft_serial = ZTP-93AF21
model = EDGE-GW-100

```
  firmware = v1.0
  nonce = 984312

[AUTH] Registration successful
[AUTH] Assigned tenant: cruise_ops

Step 6: Configuration Provisioning
The server sends configuration data in chunks.

Example Configuration:
{
  "interfaces": {
    "wan": "eth0",
    "lan": "eth1"
  },
  "mtu": 1500,
  "keepalive_interval": 20
}

[PROVISION] Receiving config (chunk 1/3)
[PROVISION] Receiving config (chunk 2/3)
[PROVISION] Receiving config (chunk 3/3)
[PROVISION] Config checksum verified

Step 7: Firmware Upgrade
The server requires a firmware upgrade.

[UPGRADE] Current: v1.0 -> Target: v1.1
[UPGRADE] Upgrade complete, restarting device

Step 8: Resume After Restart
After reboot, the device resumes provisioning without duplicate registration.

[BOOT] Restart detected
[UDP] Session resumed (session_id=7421)

Step 9: Validation and Operational Mode
Final validation is performed before entering operational state.

[VALIDATION] Configuration present
[VALIDATION] Firmware v1.1 verified
[STATE] Device is OPERATIONAL
```

Bonus: Handling Packet Loss & Out-of-Order Delivery (Advanced Networking)

This section is optional, just a bonus section and is intended to test deeper networking knowledge.

In real-world networks, especially over wireless, satellite, or congested links, UDP packets may be lost, delivered out of order, or duplicated. For this bonus, assume the network is unreliable by design.

Bonus Network Assumptions

- Packet loss: 10–20%
- Packet reordering is possible
- Duplicate packets may be received

Bonus Requirements

1. Sequence Numbers: Every UDP message must include a sequence number.
2. Acknowledgements & Retransmissions: Messages must be acknowledged and retransmitted on timeout.
3. Idempotent Processing: Duplicate messages must not cause duplicate state changes.
4. Packet Loss Simulation: Introducing artificial packet loss in code is recommended.

Sample Packet Flow Diagram

```
Device                          Server
  |---- HELLO seq=1 ---------------->|
  |---- HELLO seq=2 (lost) --------->|
  |---- HELLO seq=3 ---------------->|
  |<--- ACK seq=1 -------------------|
  |<--- ACK seq=3 -------------------|
  |---- RETRY seq=2 ---------------->|
  |<--- ACK seq=2 -------------------|
```

Example Out-of-Order Handling:
Expected seq=2, received seq=3 -> buffer seq=3 until seq=2 arrives.

Sample Log Output

[UDP] Received packet seq=3 (out of order, expected=2)
[UDP] Buffered packet seq=3
[UDP] Retransmitting seq=2 (timeout)
[UDP] Duplicate packet seq=2 ignored

Bonus README Questions

1. How does your implementation handle out-of-order packets?
2. How do you prevent duplicate processing?
3. What happens if an ACK is lost but the data packet is received?
4. What tradeoffs did you make between simplicity and correctness?

Expected Submission

Your submission must include both the working code and a README file. Submissions that include only code or only documentation will be considered incomplete.

1. Working Code (Required)

The code should:
- Implement the required Zero Touch Provisioning functionality
- Be runnable or buildable as submitted
- Clearly separate device-side and server-side logic
- Use UDP sockets for all provisioning communication
- Include basic instructions for how to run or build the code

Production-level packaging is not required, but the code should be complete and executable.

2. README File (Required)

You must include a README that explains your design and implementation choices.

At a minimum, the README should cover:
- Overall architecture and control flow
- How UDP session management is implemented
- How identity, authentication, and registration are handled
- How configuration and firmware provisioning works
- How version upgrades and restarts are handled
- Known limitations or tradeoffs

If you attempt the bonus section, the README should also explain:
- How packet loss and out-of-order delivery are handled
- How duplicate packets are detected and ignored
- How packet loss or reordering was simulated (e.g., netem or in-code logic)

3. Log Files (Required)

1. Server Logs
2. Client Logs